



Filtragem de Outliers em Engenharia de Programas

Como domar o Sistema Operacional e garantir o rigor estatístico ($CV < 0.15$) usando Percentis e Desvio Padrão.

O Inimigo Silencioso: Por que precisamos filtrar?

Ao medir o tempo de execução de primitivas em um processador real, o ambiente nunca é estéril. Eventos externos injetam ruído sistemático que invalida qualquer análise ingênua dos dados brutos.



Scheduler do SO

O *Scheduler* do Sistema Operacional pausa a nossa *thread* a qualquer momento, inserindo latências arbitrárias nas medições.



Garbage Collector

O *Garbage Collector* da linguagem entra em ação de forma não-determinística, bloqueando a execução por dezenas de milissegundos.



Thermal Throttling

O *Thermal Throttling* reduz o *clock* da CPU sob carga térmica, alterando a velocidade real de execução durante a coleta.

⊗ Esses eventos geram "picos" anômalos que destroem a média e inflam o Coeficiente de Variação (CV). Para a metrologia exata, o ruído deve ser cortado **antes** da homologação.

Técnica 1: O Corte Paramétrico ($Média \pm n \times \sigma$)

Esta técnica assume que o comportamento ideal do hardware segue uma **Distribuição Normal (Curva de Gauss)**. A premissa é que, na ausência de interferências, os tempos de execução se concentram simetricamente em torno de um valor central.

A Lógica

Calculamos a **média (μ)** e o **desvio padrão (σ)** dos tempos brutos coletados. Esses dois parâmetros descrevem completamente a distribuição assumida.

A Regra de Corte

Mantemos apenas as medições que caem no intervalo de $\mu - (n \times \sigma)$ até $\mu + (n \times \sigma)$. Tudo fora desse intervalo é descartado como outlier.

Uso Comum em Engenharia

Utiliza-se **$n = 2$** (retendo $\approx 95\%$ dos dados) ou **$n = 3$** (retendo $\approx 99.7\%$ dos dados), conforme a tolerância ao ruído residual.

95%

Retenção com $n=2$

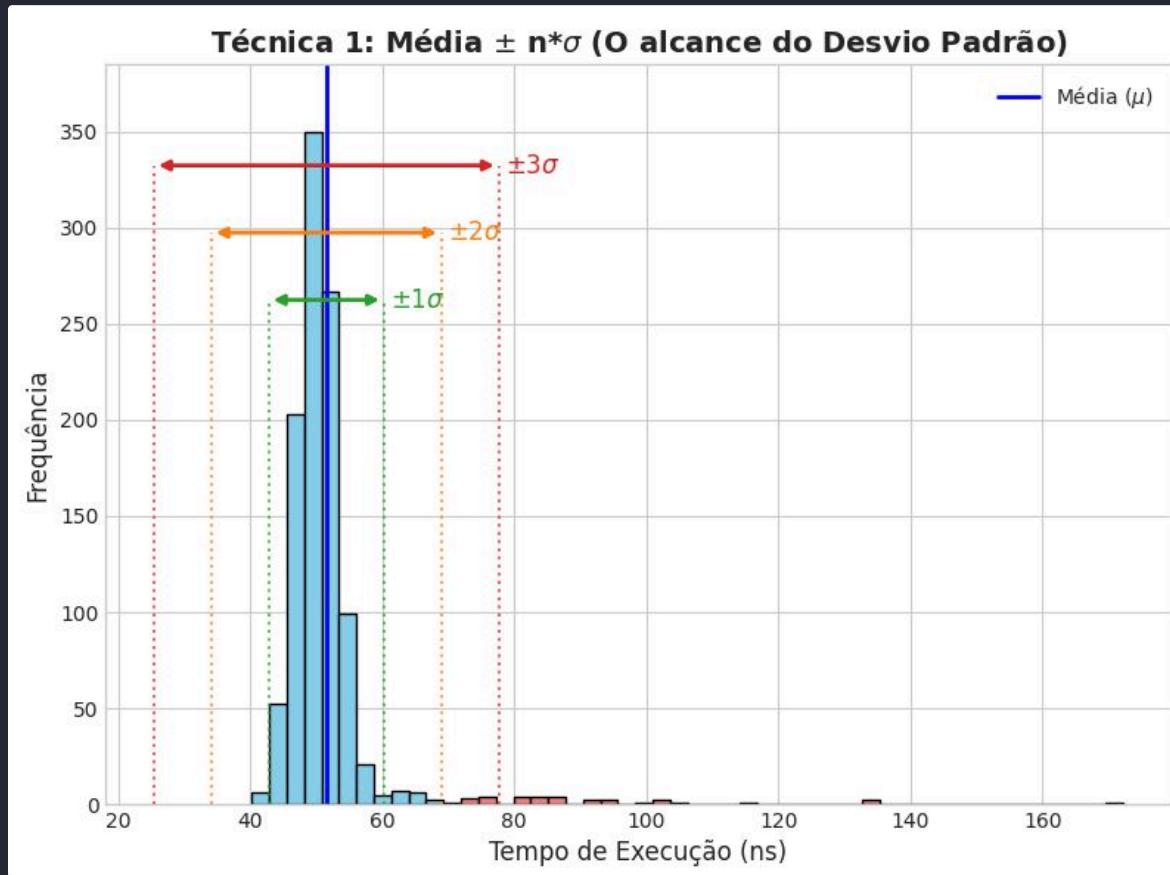
Corta as caudas mais extremas, mais agressivo.

99.7%

Retenção com $n=3$

Corte conservador, retém quase todos os dados válidos.

Visualizando o Corte por Desvio Padrão



O histograma abaixo representa a distribuição simétrica (curva de sino) dos tempos de execução brutos. A linha central tracejada marca a **média (μ)**. As duas linhas vermelhas delimitam os limiares $\mu - 3\sigma$ e $\mu + 3\sigma$.

Área Central (Válida)

Medições dentro do intervalo $\pm 3\sigma$ são consideradas representativas do comportamento real do hardware.

Caudas Externas (Outliers)

Eventos nas caudas — pausas do SO, GC, throttling — são descartados por estarem além do limiar de corte.

O Calcanhar de Aquiles do Desvio Padrão

Existe um **problema matemático severo** nesta técnica ao lidarmos com *software*: a média é "puxada" pelos próprios *outliers* que se deseja eliminar.

O Cenário do Colapso

Se o SO travar nossa medição por **2 segundos** — um valor absurdamente alto comparado aos nanossegundos normais — esse único evento gigantesco desloca a média para cima e infla o desvio padrão de forma desproporcional.

A Consequência: Filtro Cego

Com μ e σ corrompidos pelo outlier, o intervalo de corte se expande tanto que o próprio ruído que causou o problema **passa pelo filtro sem ser eliminado**. A solução exige múltiplas passagens iterativas do filtro — ineficiente e frágil.

⚠ A média e o desvio padrão são estatísticas **não-robustas**: um único valor extremo pode invalidar completamente o resultado do filtro.

Técnica 2: O Corte Robusto (Percentis)

Em vez de usar fórmulas matemáticas sensíveis a extremos, a técnica de Percentis usa a **ordenação bruta** dos dados — tornando-a imune à magnitude dos valores anômalos.



Ordenar

Ordenamos todas as medições da mais rápida para a mais lenta, sem nenhum cálculo de média ou variância.



Cortar as Pontas

Descartamos uma porcentagem fixa das extremidades. Retemos apenas os dados entre o **Percentil 5 (P5)** e o **Percentil 95 (P95)**.

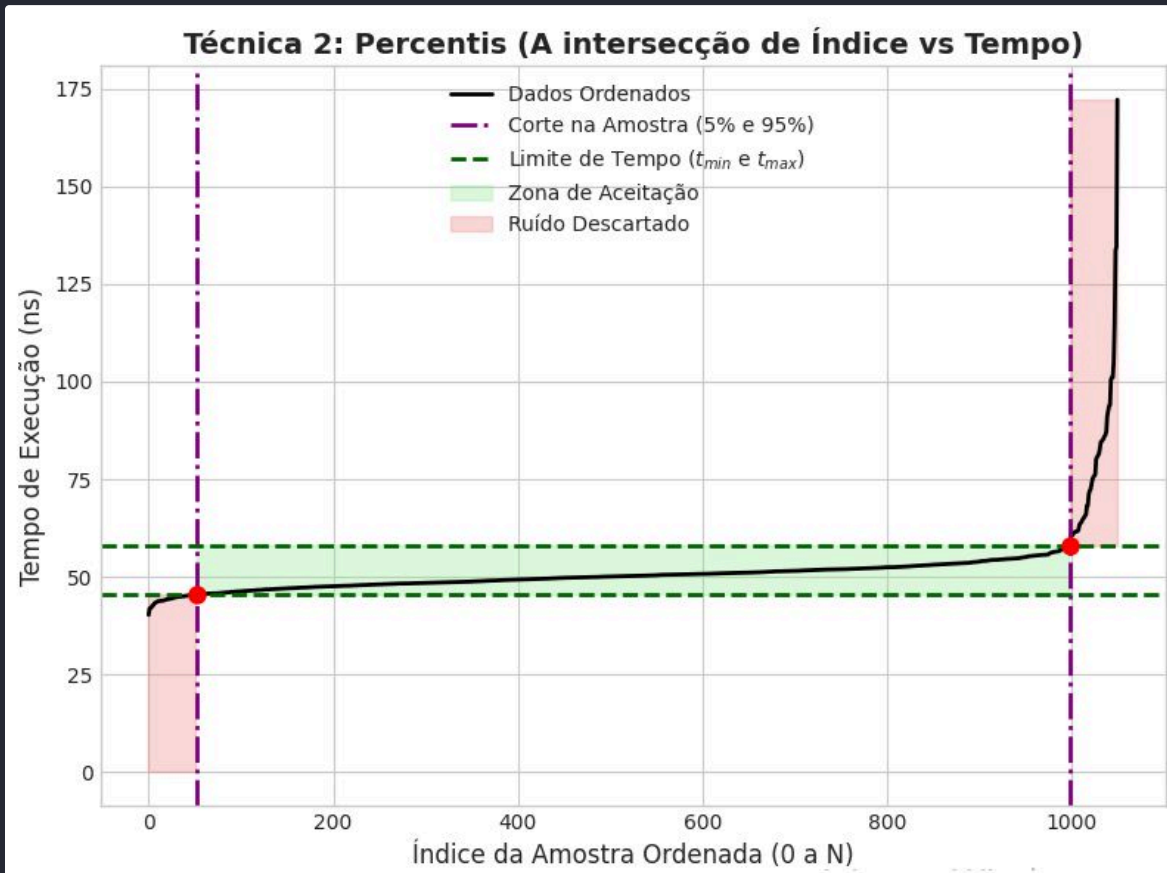


Resultado Blindado

Se o pior tempo foi 2ms ou 2 dias, não importa: ele é cortado por estar nos 5% piores, sem corromper os dados válidos.

- ✓ A vantagem estrutural dos percentis: o corte é determinado pela **posição** na ordenação, não pela **magnitude** do valor. Outliers extremos não corrompem o limiar de corte.

Visualizando o Corte por Percentis



O histograma exibe uma distribuição **assimétrica com cauda longa à direita** (*Right-Skewed*), típica de tempos de execução reais em ambientes com interferência do SO.

Linha P5 (Esquerda)

Elimina medições anormalmente rápidas — possíveis artefatos de cache quente ou erros de instrumentação.

Linha P95 (Direita)

Corta a cauda longa onde se concentram as pausas do Scheduler, GC e Thermal Throttling.

Área Verde: Dados Homologados

A região central entre P5 e P95 representa o comportamento real e repetível do hardware sob análise.

O Veredito para o Laboratório

Para a medição empírica de primitivas e funções opacas (F), a escolha da técnica de filtragem não é arbitrária — ela determina se o $CV < 0.15$ será atingido na primeira passagem ou nunca.

✗ Desvio Padrão (Paramétrico)

Assume distribuição normal simétrica. Corrompido por outliers extremos. Pode exigir múltiplas passagens. **Frágil para ambientes de SO real.**

✓ Percentis / IQR (Robusto)

Baseado em ordenação, imune à magnitude dos outliers. Garante $CV < 0.15$ na primeira passagem. **Estruturalmente superior para metrologia de software.**

Tempos de execução não são curvas de sino perfeitas

Eles formam distribuições com fortes caudas à direita (assimetria positiva), invalidando a premissa fundamental do filtro paramétrico.

O filtro de Percentis (ou IQR) é a escolha correta

É estruturalmente superior para garantir um **$CV < 0.15$** na primeira passagem, blindando a análise contra os caprichos do Sistema Operacional.